



UNIVERSIDAD TECNOLÓGICA DE PANAMÁ
CENTRO REGIONAL DE PANAMÁ OESTE
FACULTAD DE INGENIERÍA DE SISTEMAS COMPUTACIONALES
LICENCIATURA EN DESARROLLO Y GESTIÓN DE SOFTWARE

I SEMESTRE

CURSO:
DESARROLLO DE SOFTWARE VII

PROYECTO FINAL
SISTEMA DE GESTIÓN DE INVENTARIO Y FACTURACIÓN PARA UNA MINIEMPRESA

PROFESORA:
VERUSKA DE DELGADO

INTEGRANTES:
BARBA, ABDEL (8-1007-595)
BARRIA, MANUEL (8-801-841)
GUEVARA, JUAN (9-760-2463)
MARCUCCI, KEVIN (8-1036-1333)
ROMERO, GRETA (8-971-2457)

GRUPO:
9GS231

FECHA DE ENTREGA:
13 DE JULIO DE 2026

1. Introducción

La gestión del inventario y la facturación es uno de los procesos más críticos para el funcionamiento de una miniempresa, pues de ella dependen el control de existencias, la relación con los clientes y la correcta emisión de comprobantes de venta. Cuando estas tareas se realizan de forma manual o con herramientas dispersas, es común que surjan errores en el cálculo de impuestos, descuadres en el stock y dificultades para consultar el historial de operaciones. Automatizar este proceso mediante un sistema web integral permite centralizar la información y reducir el margen de error humano.

En este contexto se desarrolló el Sistema de Gestión de Inventario y Facturación para una miniempresa, una aplicación web transaccional construida bajo el patrón arquitectónico Modelo-Vista-Controlador (MVC) en PHP orientado a objetos, con persistencia de datos mediante PDO (PHP Data Objects) y procedimientos almacenados en MySQL, con el fin de mitigar los riesgos de inyección SQL y reforzar la integridad transaccional del proceso de venta.

A lo largo de las siguientes secciones se expone el objetivo general y los objetivos específicos del proyecto, el diagrama entidad-relación de la base de datos, las capturas de pantalla que evidencian el funcionamiento del sistema, la estructura de carpetas y la arquitectura de la API REST desarrollada, así como los fragmentos de código más representativos, incluyendo el enrutamiento de la API, las consultas mediante PDO y el manejo de transacciones que garantiza que no se descuente stock si una venta llega a fallar.

2. Objetivos

Objetivo general

Desarrollar un sistema web seguro y robusto para la gestión y facturación de una miniempresa, que permita un control estricto de accesos por roles, el procesamiento de ventas en tiempo real, la emisión de facturas en formato JSON y la generación de alertas tempranas ante niveles bajos de inventario.

Objetivos específicos

1. Implementar la arquitectura Modelo-Vista-Controlador (MVC) para separar la lógica de negocio, el acceso a datos y la capa de presentación del sistema.
2. Diseñar y construir una API RESTful propia que entregue reportes de ventas y facturas de forma estructurada en formato JSON, respetando los verbos HTTP y códigos de estado estándar.
3. Garantizar la persistencia segura de los datos utilizando PDO y la ejecución de procedimientos almacenados para todas las operaciones de escritura y lectura sensibles, mitigando así la inyección SQL.
4. Implementar el control de accesos mediante sesiones y cookies seguras (HttpOnly, SameSite), diferenciando los permisos entre los roles de Administrador y Vendedor.
5. Asegurar la integridad de las transacciones de venta mediante el manejo de transacciones en la base de datos (commit/rollback), de modo que no se descuente stock si la venta falla.
6. Incorporar validaciones estrictas de formularios tanto del lado del cliente como del servidor, junto con un manejo estructurado de excepciones.

3. Diagrama Entidad-Relación (DER)

A continuación, se presenta el Diagrama Entidad-Relación (DER) que modela la estructura relacional de la base de datos del sistema. El modelo está compuesto por seis entidades principales: Roles, Usuarios, Clientes, Productos, Ventas y Detalle_Ventas, relacionadas entre sí mediante llaves foráneas que garantizan la integridad referencial.

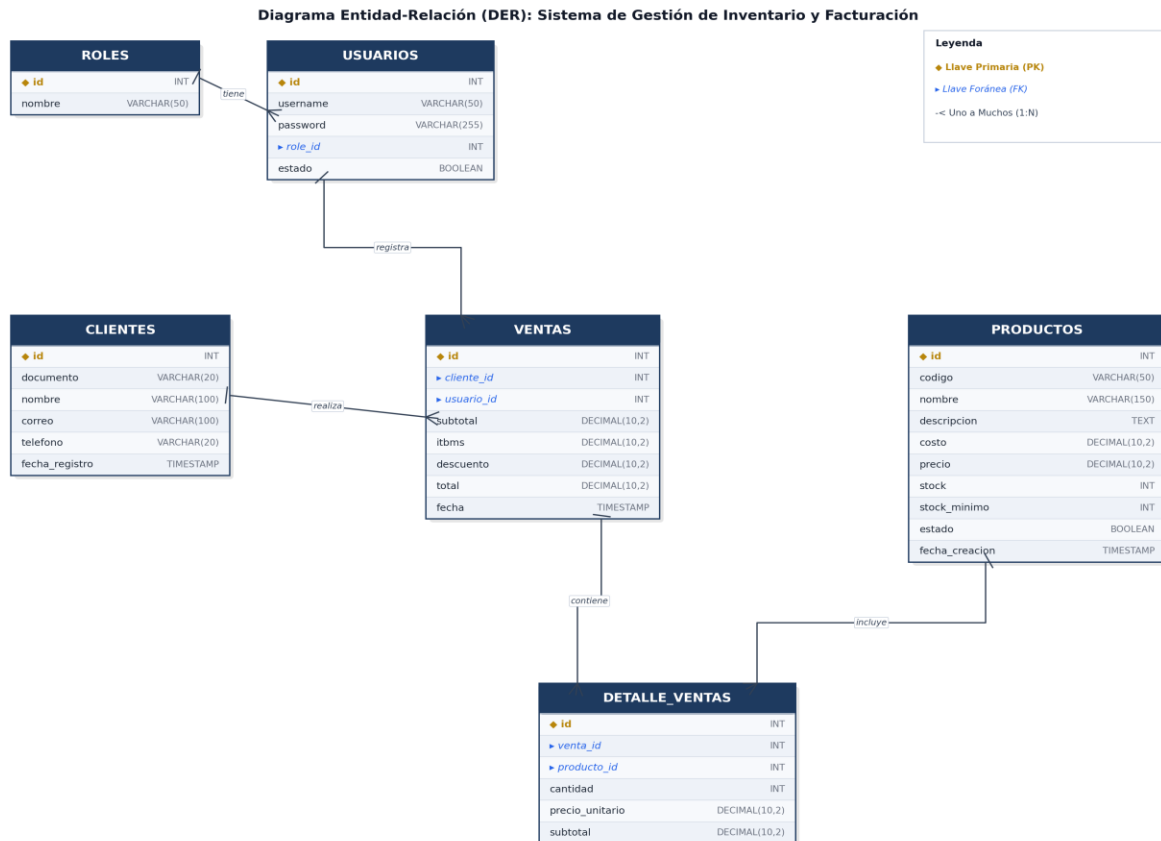


Figura 1. Diagrama Entidad-Relación del Sistema de Inventario y Facturación.

Descripción de las entidades

- **Roles:** cataloga los perfiles de acceso del sistema (Administrador, Vendedor).
- **Usuarios:** almacena las credenciales de acceso y referencia el rol asignado (role_id).
- **Clientes:** registra los datos de las personas o empresas a quienes se les emite una factura.
- **Productos:** contiene el catálogo con costos, precios, existencias (stock) y umbral mínimo de inventario (stock_minimo).
- **Ventas:** representa la cabecera de cada transacción, con los montos de subtotal, ITBMS, descuento y total, además de las referencias al cliente y al usuario que la registró.
- **Detalle_Ventas:** contiene las líneas de productos vendidos en cada venta, con cantidad, precio unitario y subtotal por línea.

4. Capturas de Pantalla de la Interfaz de Usuario

En esta sección se presentan las capturas de pantalla que evidencian el funcionamiento del sistema en ejecución. Cada recuadro indica exactamente qué pantalla del sistema debe capturarse y pegarse en el espacio correspondiente.

4.1 Inicio de Sesión

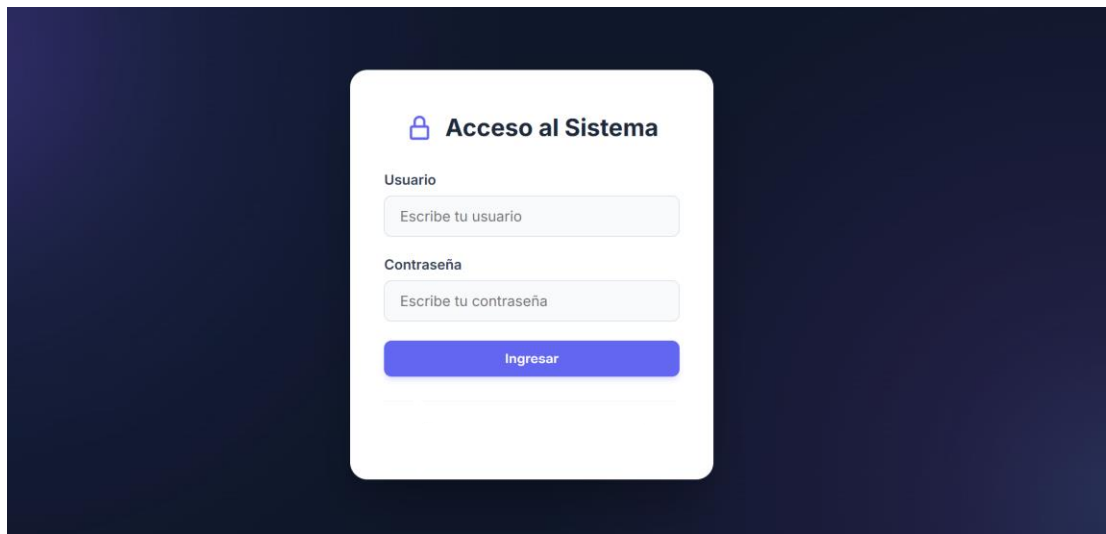


Figura 2. Pantalla de inicio de sesión del sistema, en la que el usuario ingresa sus credenciales (usuario y contraseña) para acceder según su rol asignado (Administrador o Vendedor).

4.2 Panel Principal (Dashboard)

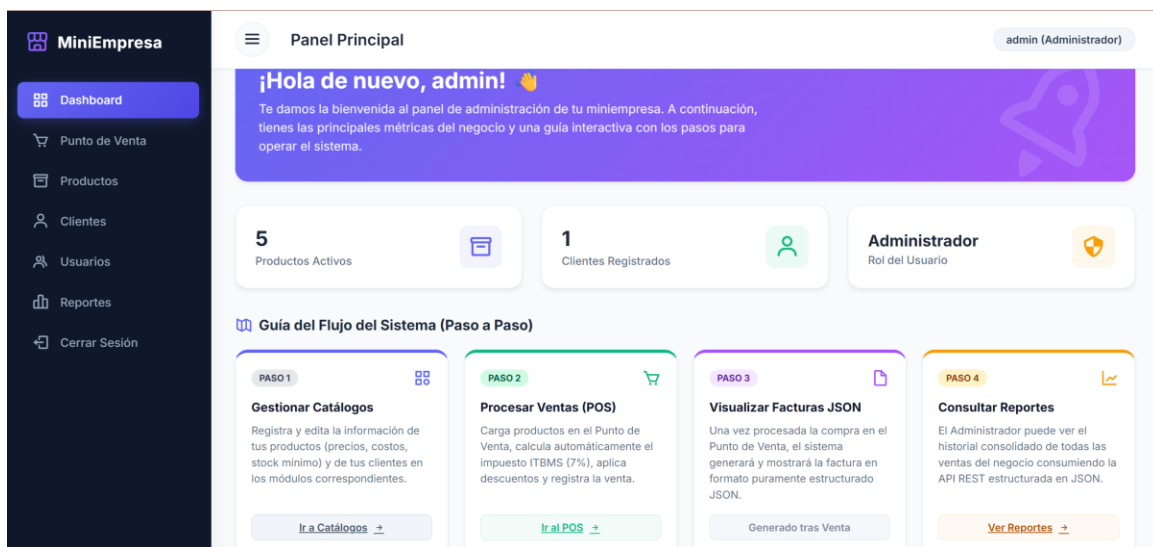


Figura 3. Panel principal del sistema tras iniciar sesión, con accesos directos a los módulos de Catálogos, Punto de Venta (POS), Facturas JSON y Reportes, además de las alertas de stock mínimo cuando corresponde.

4.3 Módulo de Productos

MiniEmpresa

Panel Principal

admin (Administrador)

Catálogo de Productos

+ Nuevo Producto

Activos 5 Inactivos 0

Código	Nombre	Costo	Precio	Stock	Stock Mínimo	Estado	Acciones
P005	Cable HDMI 2m	\$2.00	\$10.00	100	20	Activo	Editar Desactivar
P001	Laptop Dell XPS 13	\$800.00	\$1,200.00	10	3	Activo	Editar Desactivar
P004	Monitor Samsung 24"	\$100.00	\$180.00	8	4	Activo	Editar Desactivar
P002	Mouse Inalámbrico Logitech	\$10.00	\$25.00	50	10	Activo	Editar Desactivar
P003	Teclado Mecánico	\$30.00	\$60.00	15	5	Activo	Editar Desactivar

Figura 4. Catálogo de productos del sistema, donde se gestionan el código, nombre, costo, precio de venta, stock actual y stock mínimo de cada artículo (operaciones CRUD).

+ Nuevo Producto

Código de Barras / Referencia:
Ej. P006

Nombre del Producto:
Ej. Mouse Inalámbrico

Descripción corta (opcional):
Escribe detalles del producto...

Costo (\$): 0.00 Precio de Venta (\$): 0.00

Stock Actual: 0 Stock Mínimo: 5

[Cancelar](#) [Guardar](#)

Figura 4b. Modal "Nuevo Producto" del catálogo, con el formulario para registrar un producto nuevo: código de barras/referencia, nombre, descripción, costo, precio de venta, stock actual y stock mínimo, validados antes de guardar.

4.4 Módulo de Clientes

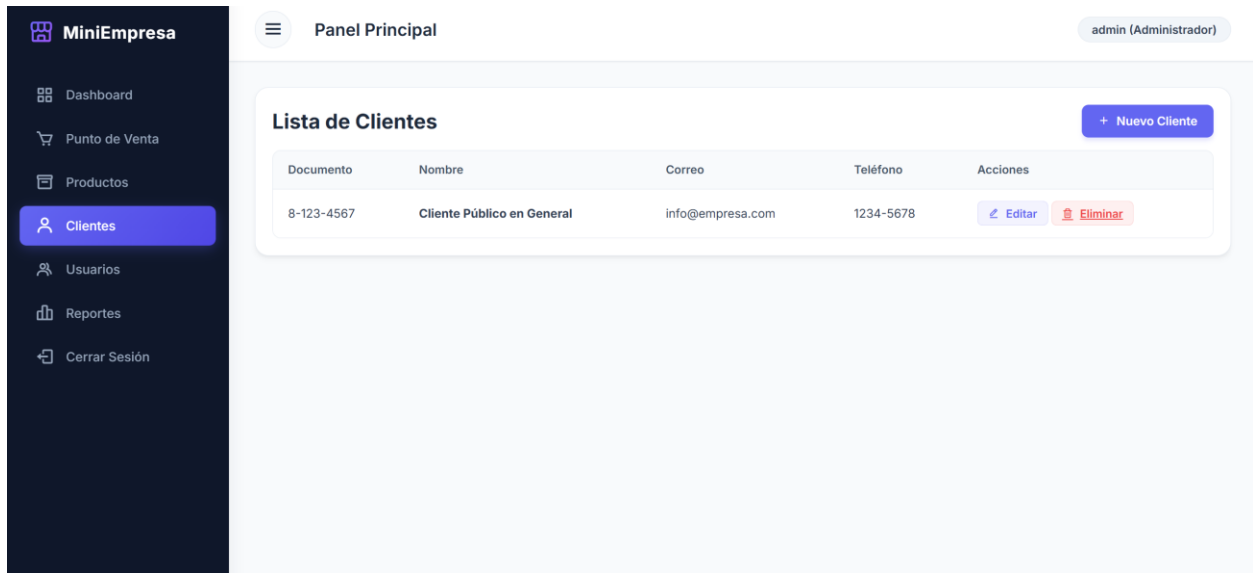


Figura 5. Módulo de gestión de clientes, donde se registran y consultan los datos de las personas o empresas a las que se les emite una factura (documento, nombre, correo y teléfono).

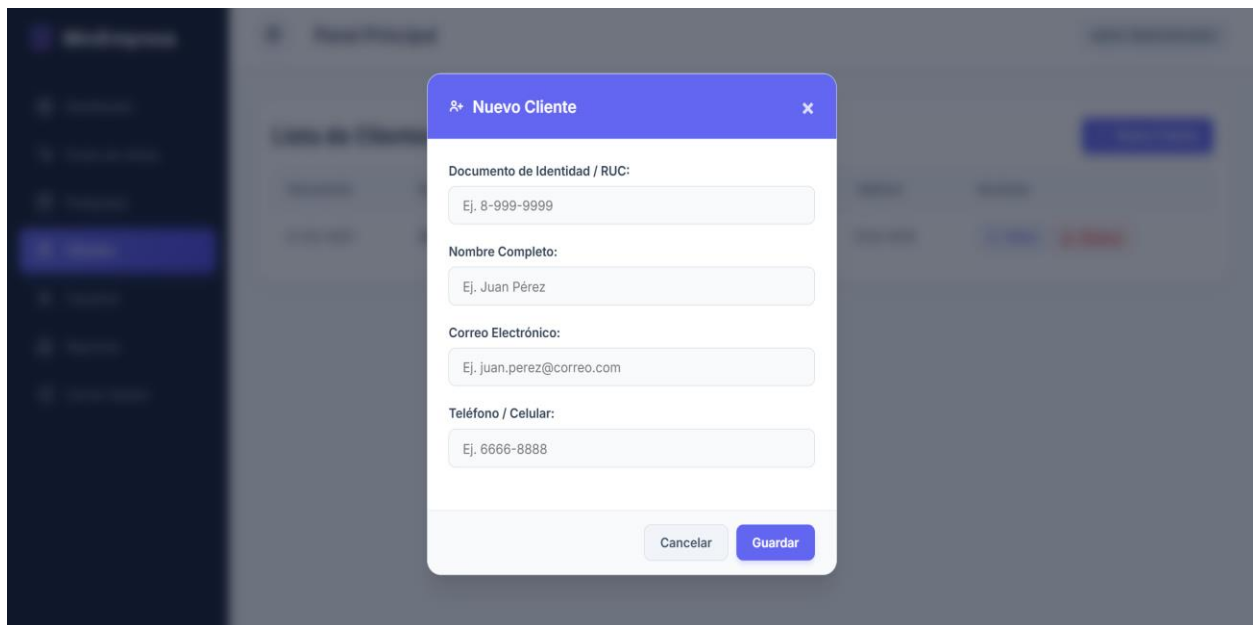


Figura 5b. Modal "Nuevo Cliente" del módulo de clientes, con el formulario para registrar un cliente: documento de identidad/RUC, nombre completo, correo electrónico y teléfono/celular.

4.5 Módulo de Punto de Venta (POS)

MiniEmpresa

Dashboard | **Punto de Venta** | Productos | Clientes | Usuarios | Reportes | Cerrar Sesión

Panel Principal admin (Administrador)

Paso 1: Cliente Busca y selecciona el cliente receptor.

Paso 2: Productos Busca en el catálogo y haz clic en (+) para añadir.

Paso 3: Cantidad Ajusta unidades con (+/-) en el detalle.

Paso 4: Cobro Ingresa descuento y procesa la venta.

1 CLIENTE

Buscar cliente por nombre o documento: Cliente Público en General (8-123-4567)

2 Catálogo de Productos Buscar por nombre o código...

ID	Producto	Precio	Stock	Acción
P001	Laptop Dell XPS 13	\$1,200.00	10 disp.	+
P002	Mouse Inalámbrico Logitech	\$25.00	50 disp.	+
P003	Teclado Mecánico	\$60.00	15 disp.	+
P004	Monitor Samsung 24"	\$180.00	8 disp.	+
P005	Cable HDMI 2m	\$10.00	100 disp.	+

3 DETALLE DE LA VENTA

Producto	Cant.	Total
Teclado Mecánico \$60.00 c/u	- 1 +	\$60.00

4 TOTALES Y CAJA

Subtotal: \$240.00

Descuento (\$): 0

ITBMS (7%): \$16.80

Total Neto: \$256.80

Procesar Venta

Figura 6. Módulo de Punto de Venta (POS), donde el vendedor agrega productos al carrito y el sistema calcula automáticamente el subtotal, el ITBMS, el descuento y el total de la venta.

4.6 Factura Generada en Formato JSON

Venta Exitosa

Factura JSON Generada:

```
{
  "success": true,
  "venta_id": "6",
  "data": {
    "venta_id": 6,
    "cliente": {
      "nombre": "Cliente Público en General",
      "documento": "8-123-4567",
      "telefono": "1234-5678"
    },
    "vendedor": "admin",
    "subtotal": "25.00",
    "itbms": "1.75",
    "descuento": "0.00",
    "total": "26.75",
    "fecha": "2026-07-12 14:01:58",
    "detalles": [
      {
        "id": 11,
        "codigo": "P002",
        "producto": "Mouse Inalámbrico Logitech",
        "cantidad": 1,
        "precio_unitario": "25.00",
        "subtotal": "25.00"
      }
    ]
  }
}
```

Ver/Imprimir Factura PDF

Entendido

Figura 7. Modal de "Venta Exitosa" desplegado al finalizar una venta, mostrando la factura generada en formato JSON con los datos del cliente, el vendedor, los montos calculados y el detalle de productos vendidos.

4.7 Reporte de Ventas

Reporte Histórico de Ventas
Datos consultados de forma asíncrona mediante la API REST del sistema.

API REST Activa

ID Venta	Cliente	Vendedor	Subtotal	Descuento	ITBMS (7%)	Total Neto	Fecha	Acción
#6	Cliente Público en General	admin	\$25.00	\$0.00	\$1.75	\$26.75	2026-07-12 14:01:58	Factura PDF
#5	Cliente Público en General	admin	\$240.00	\$0.00	\$16.80	\$256.80	2026-07-12 13:52:22	Factura PDF
#4	Cliente Público en General	admin	\$240.00	\$0.00	\$16.80	\$256.80	2026-07-12 13:50:44	Factura PDF
#3	Cliente Público en General	admin	\$240.00	\$0.00	\$16.80	\$256.80	2026-07-12 13:46:31	Factura PDF
#2	Cliente Público en General	admin	\$240.00	\$0.00	\$16.80	\$256.80	2026-07-12 13:44:40	Factura PDF
#1	Cliente Público en General	admin	\$1260.00	\$0.00	\$88.20	\$1348.20	2026-07-12 13:44:08	Factura PDF

Figura 8. Reporte histórico de ventas consumido a través de la API REST del sistema, mostrando cliente, vendedor, subtotal, ITBMS, descuento, total y fecha de cada transacción.

4.8 Gestión de Usuarios (Rol Administrador)

Gestión de Usuarios

Nuevo Usuario

Usuario	Rol	Estado	Acciones
admin	Administrador	Activo	Editar
vendedor1	Vendedor	Activo	Editar Desactivar

Figura 9. Módulo de gestión de usuarios (visible solo para el rol Administrador), donde se administran las cuentas de acceso, sus roles y su estado (activo/inactivo).



The image shows a modal window titled "Nuevo Usuario" (New User) with a close button (X) in the top right corner. The modal contains a form with three main sections: "Nombre de Usuario:" (Username) with a text input field containing "Ej. jperez"; "Contraseña:" (Password) with a text input field and a note "Mínimo 4 caracteres." (Minimum 4 characters.); and "Rol asignado:" (Assigned Role) with a dropdown menu showing "-- Seleccionar Rol --". At the bottom of the modal are two buttons: "Cancelar" (Cancel) and "Guardar" (Save).

Figura 9b. Modal "Nuevo Usuario" del módulo de gestión de usuarios, con el formulario para crear una cuenta de acceso: nombre de usuario, contraseña (mínimo 4 caracteres) y rol asignado (Administrador o Vendedor).

5. Estructura de Carpetas, Arquitectura de la API y Tecnologías Usadas

5.1 Tecnologías Utilizadas

Backend	PHP 8+ (Programación Orientada a Objetos, sin frameworks).
Frontend	HTML5, CSS3 y JavaScript (Vanilla) con Fetch API para peticiones asíncronas.
Base de Datos	MySQL, con acceso mediante PDO y procedimientos almacenados.
Seguridad	Hasheo de contraseñas con password_hash() / PASSWORD_BCRYPT, sesiones con cookies HttpOnly y SameSite=Strict, sentencias preparadas para mitigar inyección SQL.
Formato de intercambio	JSON, tanto para las respuestas de la API REST como para las facturas y reportes.

5.2 Estructura de Carpetas

El proyecto se organiza siguiendo el patrón MVC, separando de forma clara las responsabilidades de cada capa:

```

/
├── api/
│   └── index.php           Enrutador de la API REST (respuestas JSON)
├── config/
│   └── Database.php       Configuración de la conexión PDO
├── controllers/
│   ├── AuthController.php Inicio y cierre de sesión
│   ├── ClientController.php CRUD de clientes
│   ├── DashboardController.php Panel principal
│   ├── ProductController.php CRUD de productos
│   ├── SaleController.php Registro de ventas (POS) y reportes
│   └── UserController.php  CRUD de usuarios y roles
├── database/
│   └── schema.sql         Tablas y procedimientos almacenados
├── models/
│   ├── Cliente.php
│   ├── Producto.php
│   ├── Usuario.php
│   └── Venta.php          Lógica transaccional (commit / rollback)
├── public/
│   ├── css/style.css
│   └── js/app.js           Lógica asíncrona (Fetch API, carrito POS)
├── views/
│   ├── auth/, dashboard/, productos/, clientes/, pos/, reportes/, usuarios/
│   └── layout/            header, footer y modal base
└── index.php              Enrutador principal (MVC)

```

5.3 Arquitectura de la API REST

La API se implementa en `api/index.php` como un sub-enrutador independiente del enrutador principal (`index.php`). Todas sus respuestas se entregan exclusivamente en formato JSON, con las cabeceras `Content-Type: application/json` y códigos de estado HTTP estándar (200 para éxito, 404 para recursos no encontrados y 405 para métodos no permitidos).

Método	Endpoint	Descripción
GET	<code>api/index.php?endpoint=inventario/alertas</code>	Devuelve los productos cuyo stock es menor o igual al stock mínimo configurado.
GET	<code>api/index.php?endpoint=ventas/reportes</code>	Retorna el historial completo de ventas registradas en el sistema.
GET	<code>api/index.php?endpoint=facturas/{id}</code>	Genera la representación JSON de una factura específica, incluyendo cliente, vendedor y detalle de productos.

Internamente, cada endpoint invoca al modelo correspondiente (Producto o Venta), el cual ejecuta un procedimiento almacenado a través de PDO y retorna los resultados como un arreglo asociativo, que finalmente se serializa con `json_encode()`.

6. Capturas de Bloques de Código Clave

A continuación, se documentan los fragmentos de código más representativos del sistema: el enrutamiento de la API, la definición de endpoints y las consultas mediante PDO, incluyendo el uso de procedimientos almacenados y el manejo de transacciones.

6.1 Enrutador y Controlador de la API (api/index.php)

```
$method = $_SERVER['REQUEST_METHOD'];
$endpoint = $_GET['endpoint'] ?? '';
$db = (new Database())->getConnection();

if ($method === 'GET') {
    if ($endpoint === 'inventario/alertas') {
        $productoModel = new Producto($db);
        $alertas = $productoModel->obtenerAlertasStock();

        http_response_code(200);
        echo json_encode([
            "success" => true,
            "count"   => count($alertas),
            "data"    => $alertas
        ]);
        exit;
    }

    if (preg_match('/facturas\/(\d+)/', $endpoint, $matches)) {
        $venta_id = $matches[1];
        $ventaModel = new Venta($db);
        $detalle = $ventaModel->detalleFactura($venta_id);

        if ($detalle) {
            http_response_code(200);
            echo json_encode(["success" => true, "venta_id" => $venta_id, "data" => $detalle]);
        } else {
            http_response_code(404);
            echo json_encode(["success" => false, "mensaje" => "Factura no encontrada."]);
        }
        exit;
    }
}
```

Fragmento real de api/index.php: enrutamiento por endpoint y respuesta JSON con códigos HTTP.

6.2 Persistencia Transaccional con PDO y Procedimientos Almacenados (models/Venta.php)

```
public function registrarVenta($cliente_id, $usuario_id, $subtotal, $itbms, $descuento, $total,
$detalles) {
    try {
        $this->conn->beginTransaction();

        // 1. Crear cabecera de la venta mediante procedimiento almacenado
        $query = "CALL sp_crear_venta(:cliente_id, :usuario_id, :subtotal, :itbms, :descuento,
:total, @p_venta_id)";
        $stmt = $this->conn->prepare($query);
```

```

$stmt->bindParam(":cliente_id", $cliente_id);
$stmt->bindParam(":usuario_id", $usuario_id);
$stmt->bindParam(":subtotal", $subtotal);
$stmt->bindParam(":itbms", $itbms);
$stmt->bindParam(":descuento", $descuento);
$stmt->bindParam(":total", $total);
$stmt->execute();

$stmt_id = $this->conn->query("SELECT @p_venta_id AS venta_id");
$venta_id = $stmt_id->fetch(PDO::FETCH_ASSOC)['venta_id'];

// 2. Insertar cada línea de detalle (el SP valida y descuenta stock)
$query_detalle = "CALL sp_crear_detalle_venta(:venta_id, :producto_id, :cantidad,
:precio_unitario, :subtotal)";
$stmt_det = $this->conn->prepare($query_detalle);

foreach ($detalles as $det) {
    $stmt_det->bindParam(":venta_id", $venta_id);
    $stmt_det->bindParam(":producto_id", $det['producto_id']);
    $stmt_det->bindParam(":cantidad", $det['cantidad']);
    $stmt_det->bindParam(":precio_unitario", $det['precio']);
    $stmt_det->bindParam(":subtotal", $det['subtotal']);
    $stmt_det->execute(); // Si no hay stock, el SP lanza un error
}

$this->conn->commit();
return $venta_id;
} catch (Exception $e) {
    $this->conn->rollBack(); // No se descuenta stock si la venta falla
    throw $e;
}
}

```

Fragmento real de models/Venta.php: transacción atómica con commit/rollback.

6.3 Procedimiento Almacenado con Validación de Stock (database/schema.sql)

```

CREATE PROCEDURE sp_crear_detalle_venta(
    IN p_venta_id INT, IN p_producto_id INT, IN p_cantidad INT,
    IN p_precio_unitario DECIMAL(10,2), IN p_subtotal DECIMAL(10,2)
)
BEGIN
    DECLARE v_stock_actual INT;
    SELECT stock INTO v_stock_actual FROM productos WHERE id = p_producto_id;

    IF v_stock_actual < p_cantidad THEN
        SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Stock insuficiente para el producto';
    ELSE
        INSERT INTO detalle_ventas (venta_id, producto_id, cantidad, precio_unitario, subtotal)
        VALUES (p_venta_id, p_producto_id, p_cantidad, p_precio_unitario, p_subtotal);

        UPDATE productos SET stock = stock - p_cantidad WHERE id = p_producto_id;
    END IF;
END

```

Fragmento real de database/schema.sql: el procedimiento rechaza la operación si no hay stock suficiente.

6.4 Consumo Asíncrono de la API (public/js/app.js)

```

async function checkStockAlerts() {
  try {
    const response = await fetch('api/index.php?endpoint=inventario/alertas');
    const result = await response.json();

    if (result.success && result.count > 0) {
      displayStockToasts(result.data);
      const list = result.data.map(p => `${p.nombre} (Stock actual: ${p.stock})`);
      showModal('¡Alerta de Inventario!', 'Productos bajos en stock:', 'danger', list);
    }
  } catch (err) {
    console.error('Error en checkStockAlerts:', err);
  }
}

```

Fragmento real de public/js/app.js: consumo de la API propia mediante Fetch API.

6.5 Validación de Formularios (Cliente y Servidor)

Validación del lado del cliente (HTML5, views/productos/index.php):

```





```

Validación del lado del servidor (controllers/ProductController.php):

```

if (empty($codigo) || empty($nombre)) {
  $_SESSION['error'] = "El código y el nombre del producto son obligatorios.";
  header("Location: index.php?action=productos"); exit;
}
if ($costo < 0 || $precio < 0 || $stock < 0 || $stock_minimo < 0) {
  $_SESSION['error'] = "Los valores de costo, precio, stock y stock mínimo no pueden ser
negativos.";
  header("Location: index.php?action=productos"); exit;
}
if ($costo > $precio) {
  $_SESSION['error'] = "El costo no puede ser mayor que el precio de venta.";
  header("Location: index.php?action=productos"); exit;
}

```

La validación del servidor se ejecuta siempre, incluso si el navegador del cliente omite la del HTML5.